Activation Oracles

Training and Evaluating LLMs as General-Purpose Activation Explainers

Karvonen, A., Chua, J., Dumas, C., Fraser-Taliente, K., Kantamneni, S., Minder, J., Ong, E., Sen Sharma, A., Wen, D., Evans, O., & Marks, S. (2026). Activation Oracles: Training and Evaluating LLMs as General-Purpose Activation Explainers. arXiv:2512.15674 [cs.CL]. https://arxiv.org/abs/2512.15674

Huang, V., Choi, D., Johnson, D. D., Schwettmann, S., & Steinhardt, J. (2025). Predictive Concept Decoders: Training Scalable End-to-End Interpretability Assistants. arXiv:2512.15712 [cs.AI]. https://arxiv.org/abs/2512.15712

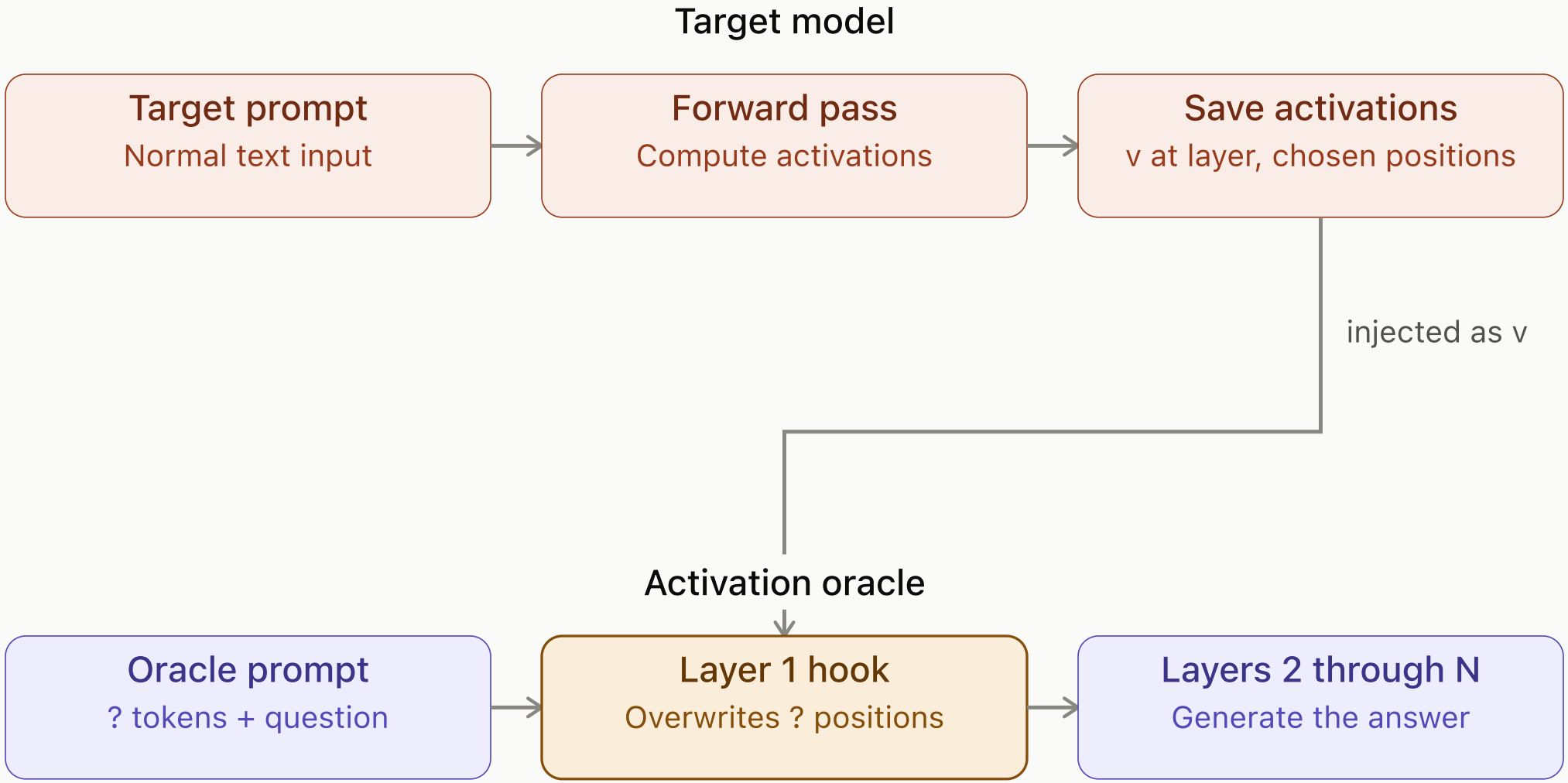
What Activation Oracles are used for

- Recovering information fine-tuned into a model — biographical facts, malign propensities — that never appears in the input text, even though the oracle was never trained on activations from that fine-tuned model
- Flagship demo: exposing both a secret goal and a secret word from a model secretly fine-tuned to play "Taboo" — purely from activations, not outputs
- **Auditing games** (Section 4) — controlled benchmarks testing whether a tool can uncover fine-tuned knowledge/behavior a model won't reveal through ordinary prompting
  - Secret-keeping: extracting hidden words, gender assumptions, or hidden instructions (Taboo, User Gender, SSC)
  - Emergent misalignment audits: describing what's driving a narrowly misaligned model, given base/fine-tuned activation differences

How Activation Oracles are trained

- **Target vs. suspect model:** "target model" is the general role — whoever's activations get captured; a "suspect model" is a target that's been fine-tuned to hide something, used specifically for the auditing evals
- **Key asymmetry:** the oracle is trained *only* on activations from the clean base model — **never** on activations from any fine-tuned/suspect model — yet still generalizes to auditing them
- **Setup:** a LoRA adapter (r=64) on the *same base architecture* as the target it will audit — trained for Llama-3.3-70B-Instruct, Gemma-2-9B-IT, Qwen3-8B, and Claude Haiku 3.5 (full fine-tune, not LoRA)
- **Training data — 3 task types mixed together:**
  - System Prompt QA (SPQA) — infer system-prompt properties from conversation activations
  - Classification — 7 binary NLP tasks (sentiment, language ID, entailment, etc.)
  - Self-supervised context prediction — predict surrounding tokens from an activation sequence, no input/target overlap
- **Scale:** ~1M examples, activations from 3 depths (25/50/75%) of the *base* model; evaluated at 50% depth; 10 H100-hrs (Qwen3-8B) / 90 H200-hrs (Llama-3.3-70B)

Injection mechanism — how target activations enter the oracle



Only the ? token positions are overwritten — every other token passes through unchanged

Steps 1 & 2 — Collect activations and build the oracle prompt

target model — Qwen3-8B, adapter="default", normal forward pass, unaware of oracle

User prompt
"Capital of France?"

→

Forward pass
layers 0–35, unchanged

→

Normal response
target unaware of oracle

↓ hook fires at layer 18 of 36 (50% depth)

activation tensor captured — shape [seq_len × 4096], one 4096-dim vector per token

↓

Step 1 Collect activations

```
collect_activations_multiple_layers(model, submodules, inputs_BL)
```

— registers forward hook on `model.model.layers[18]`

— hook stores `outputs[0]` into a dict, then raises `EarlyStopException` to stop early

↓

Step 2 Build oracle prompt + find ? positions

```
get_introspection_prefix(layer=18, num_positions=5)
```

— returns `"Layer: 18\n ? ? ? ? ? \n"` prepended to the user question

```
find_pattern_in_tokens(token_ids, " ?", n, tokenizer)
```

— scans tokenised prompt for all `" ?"` IDs, verifies count = n and consecutive, returns positions

Steps 3–5 — Package, inject and generate

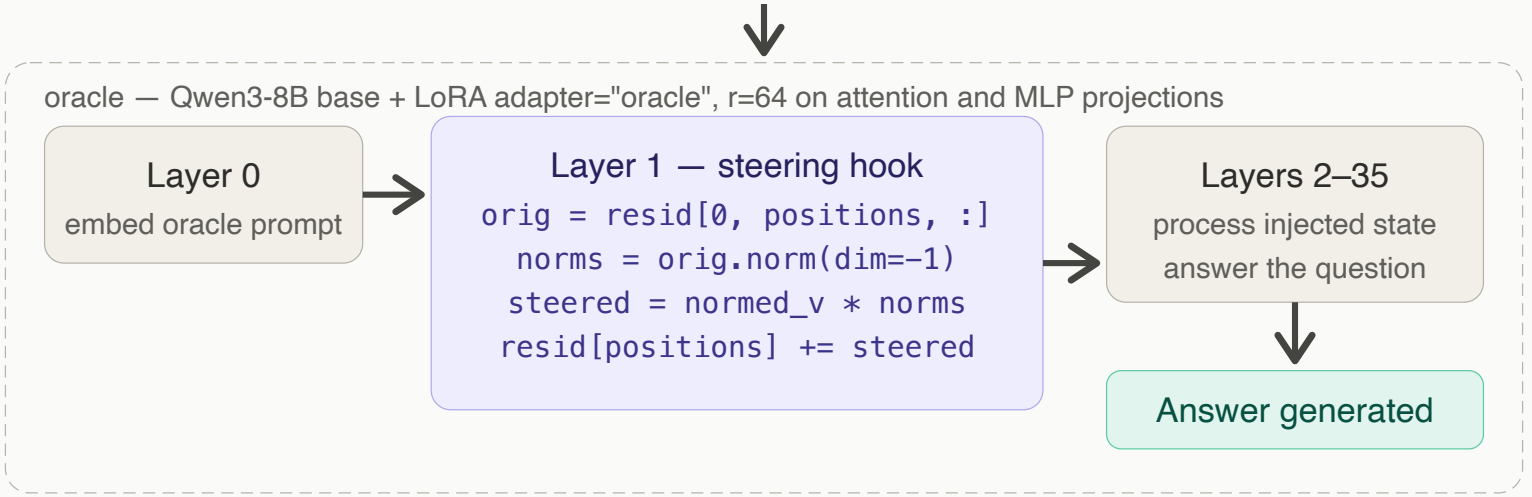
Step 3 Package into OracleInput

`create_oracle_input(prompt, layer, n, tokenizer, acts_BD)`

— returns an `OracleInput` dataclass with four fields:

`input_ids` — tokenised oracle prompt

`steering_vectors` — activations cloned to CPU, detached from graph



Layer 1 formula Norm-matching injection — Eq. 1

$$h'_i = h_i + \|h_i\| \cdot (v_i / \|v_i\|)$$

v_i — target activation, reduced to a unit vector (direction only, magnitude discarded)

h_i — oracle's own activation at that position; supplies the magnitude to rescale to

rescaled v_i is added to h_i, never replaces it — replacement caused a norm blow-up

20× at layer 0 · 100,000× at layer 1 — addition avoids this entirely

LoRA (low-rank adaptation)

- Base weight **W** (d×k) stays frozen — never updated during training
- Two small trainable matrices instead: **A** (r×k) and **B** (d×r), with r ≪ d — the oracle uses **r=64**
- Update: **ΔW = B·A**, merged at inference as **W' = W + ΔW**
- Applied to `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`
- The oracle itself is loaded as a LoRA adapter on the *same base architecture* as the target model

Worked example (d = 4096, r = 64)

- Frozen weight **W**: 4096 × 4096 ≈ **16.8M** parameters
- Trainable **A**: 64 × 4096, trainable **B**: 4096 × 64 → combined ≈ **524k** parameters
- That's roughly **32× fewer** trainable parameters than fine-tuning **W** directly
- Same idea applies per module — `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj` each get their own small **A**/**B** pair at r = 64

Norm-matching injection formula (Eq. 1)

$$\mathbf{h}'_i = \mathbf{h}_i + \|\mathbf{h}_i\| \cdot \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}$$

- $\mathbf{v}_i$ — target activation, reduced to a unit vector (direction only); $\mathbf{h}_i$ — oracle's own activation, supplies the magnitude
- **Norm** ‖v‖ = vector length, e.g. v=[3,4] → ‖v‖=5; **unit vector** v/‖v‖ = [0.6, 0.8] (length 1, same direction)
- **Rescale**: ‖h‖ · unit vector — e.g. if $\|h_i\|$=12, result is 12·[0.6,0.8]=[7.2,9.6], norm exactly 12, pointing in $\mathbf{v}_i$'s direction
- Result is **added** to $\mathbf{h}_i$, never replaces it — replacement caused **20×** (layer 0) / **100,000×** (layer 1) norm blow-up
- Why norm-match: raw activations from different sources (residual stream, SAE features, diffs) have wildly different magnitudes — rescaling to $\|\mathbf{h}_i\|$ keeps every injection at a scale the oracle already expects, so only *direction* carries new information